Accept cookies to show this video. Cookies Policy

ALLOW ALL COOKIES

Introducing AI and agentic development for Power BI

This article is about useful concepts and scenarios for integrating AI and AI agents in BI development, with specific examples for Power BI.

DOWNLOAD SAMPLE FILES

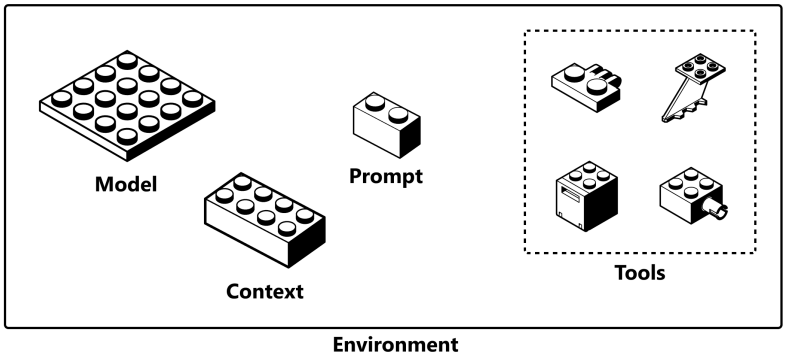
SEP 8, 2025 | Kurt Buhler AI POWER BI TABULAR

AI chatbots and agents provide interesting and useful opportunities for data professionals to become more effective and to make certain tasks more convenient. In the right scenarios, these tools can augment individuals and teams to do more and overcome technical barriers. However, you must invest time and effort to educate yourself on, set up, and get value from these tools.

In a previous article, we wrote about how model context protocol (MCP) servers can enable interesting scenarios for both conversational BI and BI development. However, to use AI effectively, it is best that you do not focus on specific models, tools, or features. This is especially true now, since they are evolving so rapidly. Instead, it makes more sense to think about the various building blocks that you combine to get the best results. None of these building blocks are sufficient alone, and all of them are required to some extent when you use AI for BI development

Consider this by using the following analogy that shows these building blocks as Lego bricks.

Building blocks of effective AI use



In general, the building blocks that you combine in your AI workflow include:

- **Large Language Model (LLM; model):** Which LLM you will use. Different models have different capabilities and require different optimization, depending on your scenario.
- **Context:** What information you will give the model to improve its results. Giving sufficient context (but not too much) is pivotal as you scale your use of AI beyond ephemeral prompts in chatbots.
- **Prompt:** The request or instructions you provide as input to an agent. Like context, writing good prompts is important to get good outputs, particularly when you use tools.
- **Tools:** How the agent will interact with its surroundings. Tools can be built-in or custom, including MCP servers, command-line interface (CLI) tools, and others.

- **Environment:** The space where the agent operates and acts; what it can and cannot see or touch. Managing your environment is pivotal to ensure safety and also limit context.

Elaborating on each of these building blocks takes significant time, effort, and examples. So, we have written a whitepaper about it for SQLBI+, including detailed explanations with four different scenarios. We introduce these scenarios below with one diagram and example, each. However, in the whitepaper, you can find over 50 different diagrams, examples, and sample files to get you started. We also have a 90-minute video walkthrough of each scenario, with examples like the following:

- Using MCP servers to interact with Fabric, run semantic-link-labs code, view mock-ups in Figma, and view published reports in the browser.
- Letting agents command and execute the Fabric CLI to take actions in Fabric.
- Using agents to build a Power BI report end-to-end.
- Delegating tasks to an asynchronous agent for development.

This article is just a high-level overview of the capabilities and considerations in each scenario. If you want a more detailed walkthrough, check out the full content (samples, whitepaper, and full video) on SQLBI+. SQLBI+ is a paid subscription service that allows us to regularly deliver more detailed, higher-proficiency content that focuses more deeply on important topics.

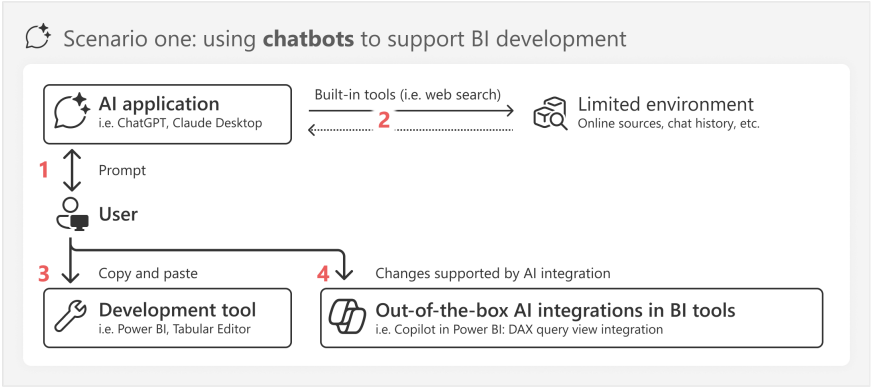
Scenarios

We currently see four different scenarios with which you can leverage AI and agents for BI development. We provide a high-level overview of each, below. Note that these scenarios are not proposing best practices or patterns to repeat. Rather, they are reporting observations that we see in the market and experience, ourselves. These scenarios are also not mutually exclusive, and they generally scale in their maturity and process complexity.

Scenario one: chatbot tools

This is the simplest scenario, where a user leverages an LLM chatbot to assist in answering questions and generating code. The chatbot has access to built-in tools and basic context provided by the user in their prompt, conversation history, or as part of a tailored experience like Copilot in Power BI. In this scenario, a user relies mainly on their prompt and out-of-the-box experiences to generate useful outputs, which they typically either download or copy for use in other tools.

The diagram for this scenario is as follows.



The diagram depicts the following processes:

1. A user engages with a chatbot via an AI application like ChatGPT, Claude Desktop, GitHub Copilot in Ask mode, and others. In the prompt, users provide context, like files, images, and other information. Most chatbots use models that have reasoning capabilities and multimodality. Users can provide basic user instructions.
2. The chatbot has built-in tools for limited engagement with its environment, including web search for retrieving up-to-date information from online sources, or some form of memory, like searching over the chat history.
3. The user typically provides code or context to the LLM, and then receives code back that they can paste into their development tool, like Power BI or Tabular Editor. The LLM might also provide outputs like generated images that the user can upload (such as canvas backgrounds or icons).
4. The user might also leverage out-of-the-box AI integrations in their BI tools, like Copilot in Power BI. An example is using Copilot to suggest DAX measures in the DAX query view, or suggesting a report page.

Here is an example of what this scenario looks like in Claude Desktop.

An example of using **chatbots** to support BI development

DAX Workdays MTD Calculation

Share

WorkdaysMTD.dax

49 lines

TEXT

Read the expression for the Workdays MTD measure and look at the image of the example query. The measure expression returns incorrect results where holidays and weekends return a wrong result <example>1/1/2016-1/4/2016 is 2303 but should be 1, 1/5/2016 should be 2, etc.</example>. Follow the logic described. Ask if you require clarification.

<logic>
* Workdays should cumulate from month start to month end
* Weekends and holidays should always have the same value as the previous workday
  * <example>1/9/2016 and 1/10/2016 should have the same value as 1/8/2016</example>
* If the month starts on a weekend or holiday, then the weekend or holiday should have the same value as the first workday that is not a holiday
  * <example>1/1/2016, 1/2/2016, 1/3/2016 should have the same value as 1/4/2016</example>
</logic>

Before you answer, search the web and check only the following references:
<references>
* **DAX Guide:** http://dax.guide/
* **Microsoft Documentation:** https://learn.microsoft.com/en-us/dax/
* **SQLBI:** https://www.sqlbi.com/articles/
</references>

Diagnosed DAX measure logic flaw for workday calculation.

Looking at the code and output, I can see the issue. The `_EndDate` calculation for non-workdays is completely wrong - it's using `REMOVEFILTERS()` which is giving you the wrong

The previous example is a bit verbose, but still quite simple. However, you can see how the user relies mostly on their prompt and some built-in tools (like web search) to get help. In this case, the user might copy the DAX code to their model to test it.

In this scenario, you can also use out-of-the-box AI integrations in BI tools, like Copilot in Power BI. An example is the Copilot experience in the DAX query view, which might support you in semantic model or report development by helping you to generate DAX measures and queries.

An example of using **Copilot** to support BI development

Run | Update model with changes (0)

```
7+   MEASURE 'Product'[Product Count] =
8+     DISTINCTCOUNT('Product'[ProductKey])
9+   // Measure to count distinct stores.
10+  MEASURE 'Store'[Store Count] =
11+    DISTINCTCOUNT('Store'[StoreKey])
12+
13+ // This query provides a basic example filtering data for a specific calendar mo
14+ // In this example, we filter to the "April 2017" month (from the 'Date' table's
15+ // and to the "Audio" product category.
16+ EVALUATE
17+   ADDCOLUMNS(
18+     SUMMARIZECOLUMNS(
19+       'Date'[Month],
20+       'Product'[Subcategory],
21+       TREATAS({ "April 2017" }, 'Date'[Year Month]),
22+       TREATAS({ "Audio" }, 'Product'[Category])
23+     ),
24+     "Customer Count", [Customer Count],
25+     "Product Count", [Product Count],
26+     "Store Count", [Store Count]
27+   )
28+   ORDER BY
29+     'Date'[Month] ASC,
30+     'Product'[Subcategory] ASC
```

Suggest a basic DAX query with measures that will count the number of customers, products, and stores. Each of the measures should be filterable by the dimensions in the model, so I can see how many customers bought a particular product, and how many stores sold a particular product (or how many customers they had in a month). Include a basic example query that filters to a particular calendar month and product category. 424/2000

Results | Result 1 of 1 | Copy

	Date[Month]	Product[Subcategory]	[Customer Count]	[Product Count]	[Store Count]
1	April	Bluetooth Headphones	5585	50	74
2	April	MP4&MP3	5585	45	74
3	April	Recording Pen	5585	20	74

The previous example produces incorrect results, but it might be a good starting point for beginners. The correct result could involve using the key columns in the *Sales* fact table and not in the dimension tables (which cannot filter each other). There are various valid use-cases and benefits for using Copilot in this experiment.

Copilot in the DAX query view can automatically read model metadata as context. It also has various pre- and post-processing steps to improve outputs compared to other, generic AI applications; however, you have less visibility and control over Copilot's capabilities, configuration, and which LLM is used.

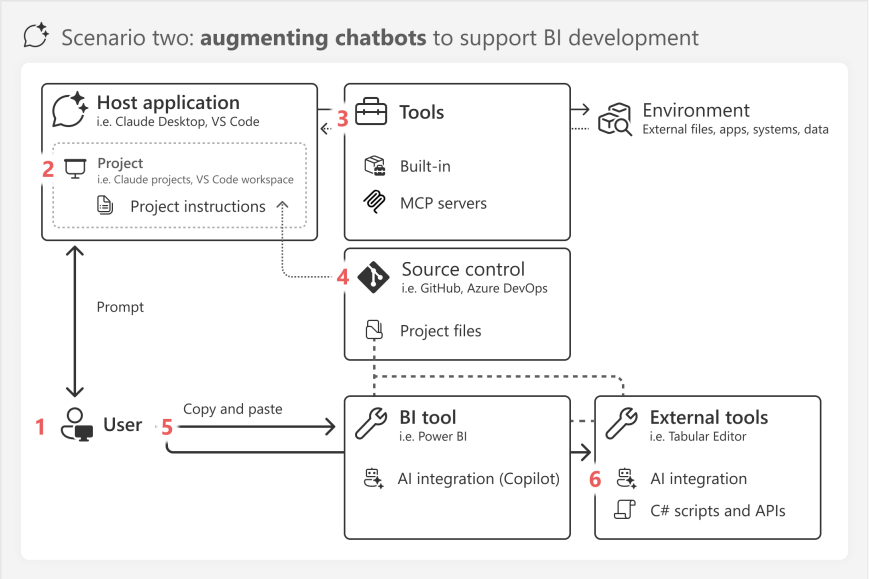
Most people are familiar with this scenario. It requires minimal setup and has limited process complexity. However, it also has limited utility and higher risk for wasting time or falling victim to LLM mistakes or fallacies.

Scenario two: Augmented chatbot experiences

In this scenario, users augment the chatbot experience by better leveraging context (such as instructions and remote repositories) and custom tools (such as MCP servers). Tools allow the chatbot to better retrieve context or even take actions in an environment. You can use MCP servers to make changes to or develop a semantic model or report. However, the MCP server tools will be limited in their capabilities, and the experience is not very convenient without a supporting user interface.

Here, there is a pre-requisite that you use Power BI Projects format (PBIP) so that the model metadata is readable and writable by both humans and AI. It is also recommended that you use remote repositories like GitHub or Azure DevOps Repos for source control, since you can connect them to AI applications.

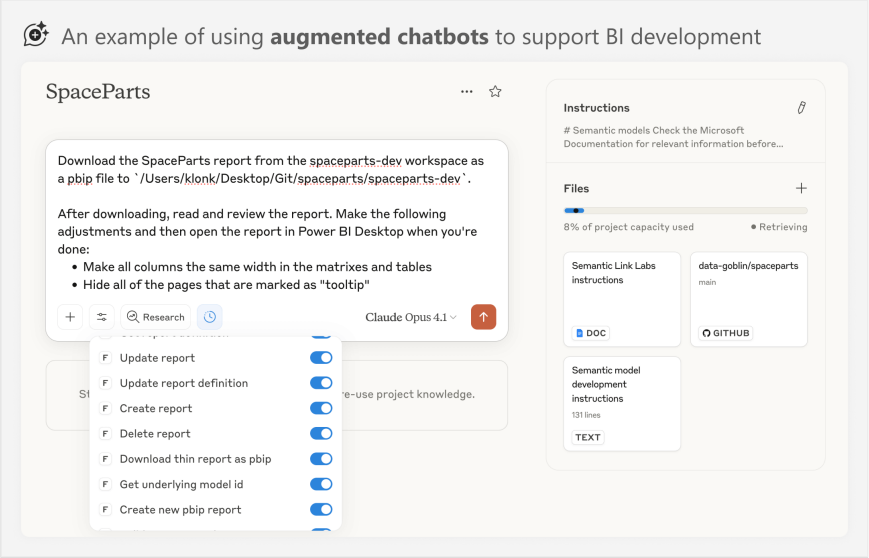
The diagram for this scenario is as follows.



The diagram depicts the following processes:

1. The user still uses a chatbot in an AI application, such as Claude Desktop or VS Code, simultaneously with BI tools and external tools, depending on their preference and workflow.
2. The user sets up a project or folder in the host application that contains special context and instructions for the chatbot.
3. The user leverages a combination of built-in tools and custom MCP servers. These could be MCP servers from official vendors, open-source MCP servers from trusted sources and extensions, or custom local MCP servers the user has built and uses for their own needs.
4. The user connects a remote GitHub repository to a Claude project or opens the cloned repository in VS Code. The LLM cannot change these files and can only read them into context.
5. Changes to models and reports are made by copying and pasting code into the BI tool or external tool.
6. Some external tools enable making modifications directly, such as C# scripts in Tabular Editor which can call and use the APIs for various providers of LLM services and products.

You can see an example below of the augmented chatbot scenario.



The example shows a Claude Desktop project where the user no longer relies on ephemeral prompts alone, but invests more in providing context via instructions and metadata. They also use custom MCP servers to provide context and tools for the model. This example is a static image that shows the different parts, but our previous article and video shows several examples of this.

This scenario requires more time and effort to create and curate both context and tools. Here, the user exercises more autonomous intent based on what they want the chatbot to do, and what kinds of outputs they expect. The scenario thus requires additional planning and critical thinking from the user.

Scenario three: Agentic development

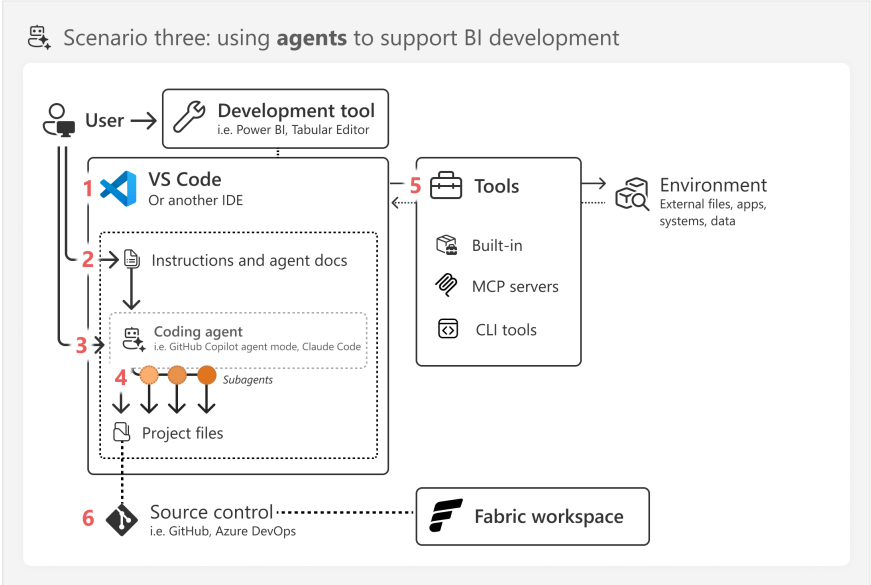
This scenario describes the use of agentic tools to help develop and manage semantic models, reports, and other artifacts or processes in your BI platform:

- In simple scenarios, you can use the agent to search and make limited changes to your model or report metadata. An example could be parameterizing Power Query steps or copying conditional formatting from one column to another in a table or matrix. This scenario is an extension of scenario two, where the user needs to provide limited instructions and tools, mainly leveraging the built-in tools to make changes.
- In more advanced scenarios, you can use the agent to facilitate larger development tasks. An example could be building an entire report from a wireframe or mock-up, before a human takes over for polishing and revisions. This advanced scenario requires a certain process maturity and consistency, and significant effort in creating and curating context. Even so, in their current state, agents require substantial supervision and orchestration.

Agents that work on Power BI metadata files (like TMDL, PBIR, and others) have a tendency to make mistakes, not only because of the sparseness of this information in training data, but also because Power BI semantic models and reports are complex.

In this scenario, the user focuses on the use of agentic coding tools either in an integrated development environment (IDE) or on a terminal scoped to a particular project. The user focuses most of their effort on orchestrating agent planning and curating the context with granular instructions, while also iteratively improving and validating agent outputs. The agent can read and write project files, and exhibits autonomous (but supervised) use of built-in tools, MCP servers, and CLI tools to gather context or take actions in its environment. This scenario can be done in combination with scenario two.

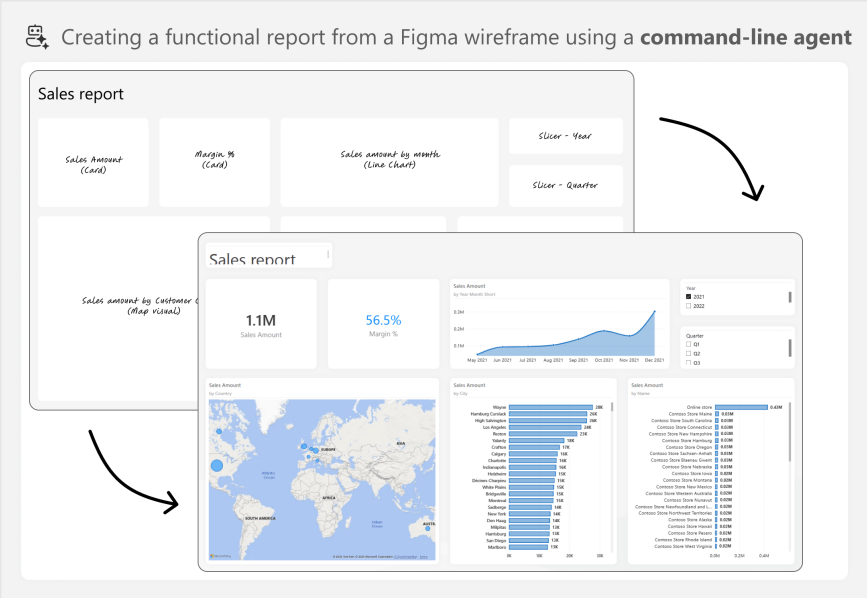
The diagram for this scenario is as follows.



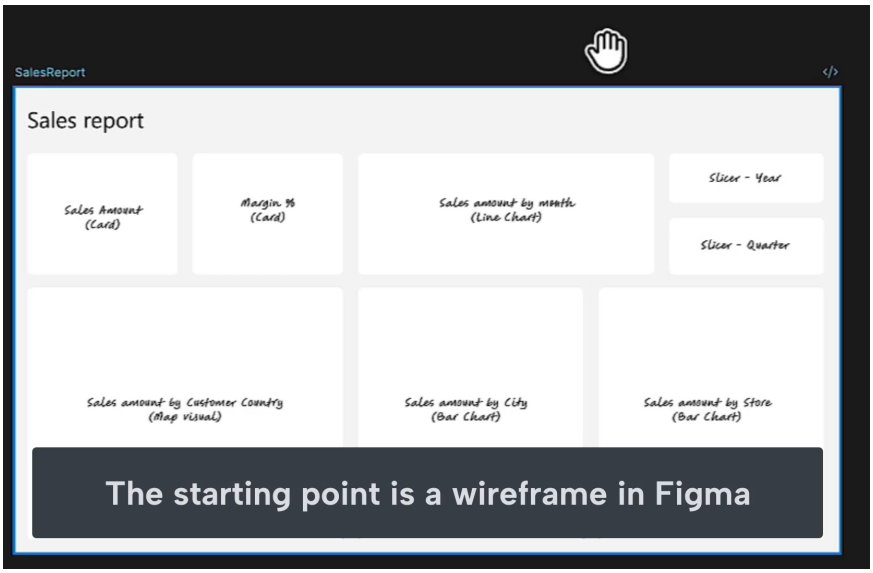
The diagram depicts the following processes:

1. The user uses agents inside of an IDE (but they could also use only a terminal for CLI agents).
2. Most of their time goes toward creating instructions, design documents, and agent documents to curate a set of rich but specific and well-written context for the agent to perform their task well. The context must accurately reflect the business and technical reality.
3. The user prompts the coding agent to help with development tasks. The bulk of report and model development is still performed by the user; it is not realistic to expect that agents will be able to handle the majority of end-to-end development tasks for Power BI any time soon.
4. The agents can read and write changes to code directly. If supported, the user can configure specialized subagents that the main orchestrator can use in parallel or in sequence for certain tasks.
5. Users configure agents to have access to built-in tools, MCP servers, and also command line tools. These tools give the agent extensive control over their own context and capabilities in the environment. More context is needed to use CLI tools successfully.
6. The user makes use of remote repositories to structure the work and restore previous versions when necessary. They can set up streamlined deployment to a Fabric workspace for testing, by using Git integration, tools, or custom deployment methods.

Here is a more sophisticated example, where a main Claude Code agent orchestrates a team of specialized subagents to build a report based upon a wireframe in Figma.



You can see this demonstrated in the below animated video. The demo is sped and cut due to size limits. You can view the full, uncut demonstration and explanation on SQLBI+, as well as the sample files used to make it:



Here is what happens in this short demo:

1. A command-line agent (Claude Code) uses an MCP server to read the Figma wireframe.
2. The user runs a re-usable prompt to plan reports.
3. The agent infers from the design and conversation history what model to use.
4. It searches for the model and maps fields to visuals by controlling the Fabric CLI.
5. It produces a plan for user review.
6. The user reviews and adjusts the plan, then approves it by starting development.
7. The agent builds a Power BI Project and delegates to a team of specialized sub-agents the building of the page and visual metadata. Subagents are executed in parallel.
8. The orchestrating agent reviews the resulting metadata against the spec and schemas.
9. The user approves the development and asks the agent to deploy to a Fabric workspace.
10. The agent publishes the report, fixes minor issues.
11. The agent controls the user's browser to view and validate the published, functional report.

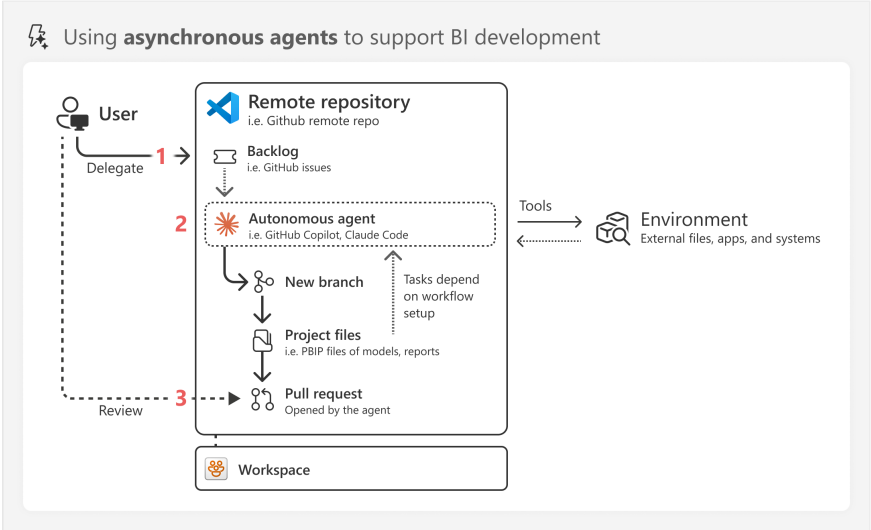
Please do not over-extrapolate this example; ***AI cannot make reports or semantic models for you.*** Rather, we use a complex example to demonstrate a variety of processes and considerations, and how an agent might contribute to the process at various points, where it is necessary and where it is helpful.

To reiterate, the example is more sophisticated. You can also use agents for simpler tasks, but the example showcases more completely what is possible. Indeed, obviously this scenario is more advanced. It takes a lot of time and effort to set up and use it properly beyond simple searches and batch changes. This is not something that “just works” and it requires a good understanding of the nuance and application of critical thinking to get value from it.

Scenario four: Asynchronous agents

This scenario involves a user delegating specific tasks to asynchronous agents for the agent to work on the task autonomously without supervision. The task could be something simple like reviewing a pull request (PR), or more sophisticated like making changes in the background. This scenario is distinct from scenario three, since the human does not orchestrate or supervise the agent’s activities directly with prompts, in a chatbot session. Rather, the human orchestrates it indirectly via instructions (and possibly an initial prompt) and then reviews the result (typically as a new PR that the agent opens).

The diagram for this scenario is as follows.



The diagram depicts the following processes:

1. The user can delegate a task to an agent via an issue backlog or a comment on an existing issue or PR.
2. The agent can start working on the task asynchronously. It has access to tools to act in its environment, and its exact workflow is configured by the user. Typically, the agent will create a new branch, modify project files, and then notify the user via a new comment in the issue or PR.
3. The user can then review the comment and the changes. If the agent made changes to project files, the user can review them in a new PR.

You can see an example of this below, where Claude Code works asynchronously on tasks described in a GitHub issue.

An example using **asynchronous agents** to support BI development

data-goblin opened 1 minute ago

Model: spaceparts-otc-full
Object: 'Products' partition
Task: Remove the filter in the partition source expression on the 'Products'[Type] filtering out "Projectiles"
Described in: tasks.md

Create sub-issue

data-goblin 1 minute ago Author

@claude please read the referenced file and implement this fix.

claude bot 1 minute ago · edited by claude Edits

Working on removing Products filter

Tasks:

- [x] Read the referenced tasks.md file to understand requirements
- [x] Locate the spaceparts-otc-full semantic model files
- [x] Find the Products partition with the filter expression
- [x] Remove the filter excluding "Projectiles"
- [ ] Commit and push changes
- [ ] Provide PR creation link

Successfully removed the `Table.SelectRows` filter! The partition now returns unfiltered data directly instead of excluding "Projectiles".

In the previous example, the user delegates a development task to Claude Code, which proceeds stepwise to work on it in the background, without human supervision. Currently, asynchronous agents have limited realistic applications beyond very finely-scoped tasks and reviews.

On security and cost

In this content and the associated whitepaper, we do not focus on particular models or tools. Rather, we focus on the high-level concepts, scenarios, and use-cases. As such, we provide no detailed information about cost or security considerations, at this time. The objective of this content is simply to provide a detailed overview of the possibilities and a realistic view of the thought, time, and effort required to apply them.

Conclusions

AI chatbots and agents provide some interesting opportunities to augment your BI workflows beyond simply copy-pasting code and generating background images of your reports. If you are willing to invest sufficient time and effort, you can set up agents that can help you with a wide range of tasks. However, doing so does require a certain process maturity, first. If you are still figuring out the basics, then adding AI and agents on top is probably not going to help you; instead, it might just make your mess more complicated.

If you are interested, please check the whitepaper at SQLBI+ for more details, guidance, and examples, including relevant sample files and full demonstrations of each scenario.

Downloads

Insert your email address and press *Download* for access to the files used in this article.

Your email address DOWNLOAD

Keep me informed about BI news and upcoming articles with a bi-weekly newsletter
(uncheck if you prefer to proceed without signing up for the newsletter)

Send me SQLBI promotions (only 1 or 2 emails per year)

By downloading the file(s), you agree to our Privacy Policy and reCAPTCHA Privacy Policy.

PUBLISHED SEP 8, 2025

WRITTEN BY
Kurt Buhler

Kurt is a business intelligence consultant and mentor. He specializes in understanding how a business works to take a user-centric approach to business intelligence... MORE

Search for more articles, blog posts, and videos SEARCH

FEB 24, 2025

Account receivable aging in Power BI

This article describes an Accounts Receivable Aging report in Power BI, and shows how to simplify a business problem using existing modeling patterns. READ MORE →

JAN 16, 2025

Optimizing incremental inventory calculations in DAX

This article describes how to optimize inventory calculations in DAX by using snapshots to avoid the computational cost of a complete running total.
READ MORE →

NOV 20, 2023

Computing accurate percentages with row-level security in Power BI

This article shows how to compute ratios when row-level security hides some of the data. If the percentage also includes the hidden rows in the comparison, you should customize the data model and the measures involved to get the right...
READ MORE →

JUL 31, 2023

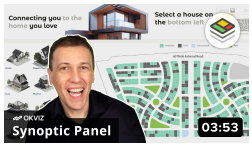
Comparing different school terms in Power BI

This article describes how to implement the comparison between school terms. The same technique can be applied to any arbitrary time periods that do not match regular months or quarters in a calendar, such as seasons or campaigns.
READ MORE →

MAR 10, 2026

Creating functions for the like for like DAX pattern

WATCH NOW →



MAR 8, 2026

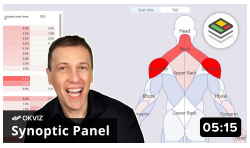
Real estate with Synoptic Panel by OKVIZ

WATCH NOW →

FEB 24, 2026

Debug DAX variables using TOJSON and TOCSV

WATCH NOW →



FEB 19, 2026

Displaying injury data on the human body by using Synoptic Panel in Power BI

WATCH NOW →

Comments

Accept cookies to show comments. Cookies Policy

ALLOW ALL COOKIES

Newsletter

Your email address SUBSCRIBE

By subscribing, you agree to our Privacy Policy and reCAPTCHA Privacy Policy.

275.8K subscribers

Get BI news and original content in your inbox every 2 weeks!

NEWSLETTER ARCHIVE →

What is next

- Mastering DAX – Mar 30, Houston
- Mastering DAX – Apr 8, New York City
- Mastering DAX – May 5, Copenhagen
- Mastering DAX – May 27, Baltimore
- Mastering DAX – Jun 10, Zürich
- Mastering DAX – Jun 15, Berlin
- Mastering DAX – Jun 30, Amsterdam

2004-2026 © SQLBI. All rights are reserved.
SQLBI is a trademark of SQLBI Corp.
Loader is a European legal entity.

PRIVACY POLICY & COOKIES →
TERMS & CONDITIONS →
CONTACT US →